Unit -2 Process management

1. Thread and process concept Inter process communication
2. (Critical-section problem,
3. solving critical section problem with busy-waiting and sleep and wakeup strategies, Semaphores,
4. Monitors)
5. Process scheduling algorithms

Unit -2 Process management

Thread and process concept Inter process communication

Threads:

A process is divide into number of light weight process, each light weight process is said to be a Thread. The Thread has a program counter (Keeps track of which instruction to execute next), registers (holds its current working variables), stack (execution History).

In the context of an operating system (OS), a thread is a fundamental unit of CPU utilization that represents a single sequence of executable instructions within a process.

Thread States:

1. bornState : A thread is justcreated.
2. readystate : The thread is waiting forCPU.
3. running : System assigns the processor to thethread.
4. sleep : A sleeping thread becomes ready after the designated sleep timeexpires.
5. dead : The Execution of the threadfinished.

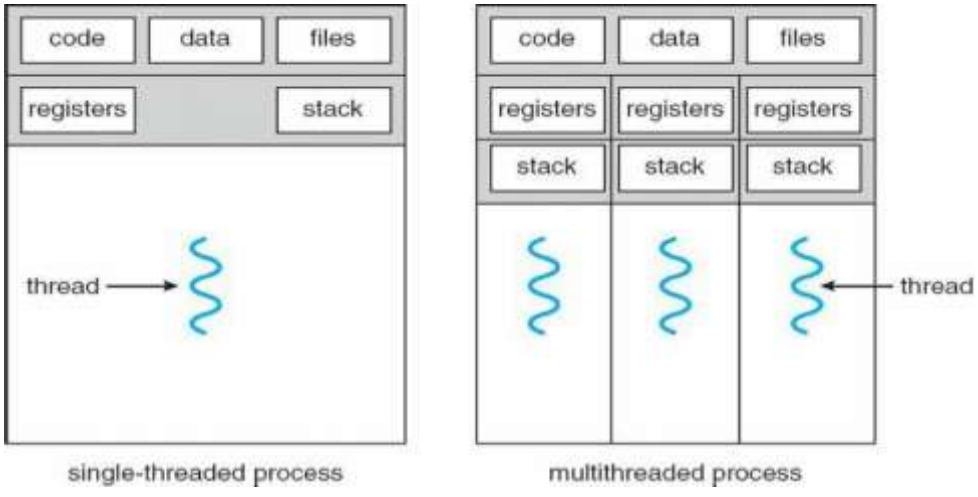
Eg: Word processor.

Typing, Formatting, Spell check, saving are threads.

Process	**Thread**
Process takes more time to create.	Thread takes less time to create.
it takes more time to complete execution & terminate.	Less time to terminate.
Execution is very slow.	Execution is very fast.
It takes more time to switch b/w two processes.	It takes less time to switch b/w two threads.
Communication b/w two processes is difficult .	Communication b/w two threads is easy.
Process can't share the same memory area.	Threads can share same memory area.
System calls are requested to communicate each other.	System calls are not required.
Process is loosely coupled.	Threads are tightly coupled.
It requires more resources to execute.	Requires few resources to execute.

Multithreading

A process is divided into number of smaller tasks each task is called a Thread. Number of Threads with in a Process execute at a time is called Multithreading. Multithreading gives best performance.



Kernels are generally multithreaded

CODE- Contains instruction
DATA- holds global variable
FILES- opening and closing files
REGISTER- contain information about CPU state
STACK-parameters, local variables, functions

Types Of Threads:

1) User Threads : Thread creation, scheduling, management happen in user space by Thread Library. user threads are faster to create and manage. If a user thread performs a system call, which blocks it, all the other threads in that process one also automatically blocked, whole process is blocked.

Advantages

- Thread switching does not require Kernel mode privileges.
- User level thread can run on any operating system.
- Scheduling can be application specific in the user level thread.
- User level threads are fast to create and manage.

Disadvantages

- In a typical operating system, most system calls areblocking.
- Multithreaded application cannot take advantage ofmultiprocessing.

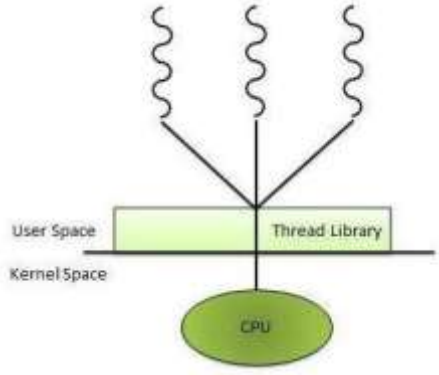
2) Kernel Threads: kernel creates, schedules, manages these threads. threads are slower, manage. If one thread in a process blocked, over all process need not be blocked.

Advantages

- Kernel can simultaneously schedule multiple threads from the same process on multiple processes.
- If one thread in a process is blocked, the Kernel can schedule another thread of the same process.
- Kernel routines themselves can multithreaded.

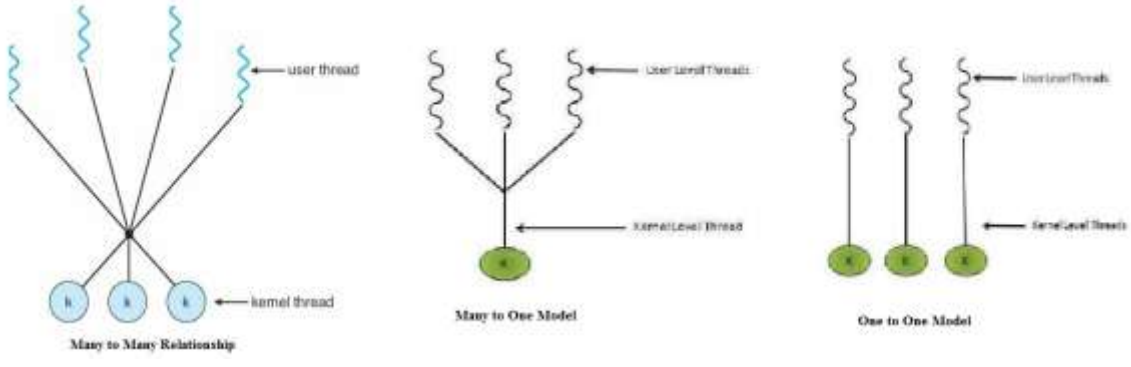
Disadvantages

- Kernel threads are generally slower to create and manage than the userthreads.
- Transfer of control from one thread to another within same process requires a mode switch to the Kernel.



Multithreading Models

Some operating system provides a combined user level thread and Kernel level thread facility. Solaris is a good example of this combined approach. In a combined system, multiple threads within the same application can run in parallel on multiple processors and a blocking system call need not block the entire process. Multithreading models are three types



Difference between User Level & Kernel Level Thread

S.N.	User Level Threads	Kernel Level Thread
1	User level threads are faster to create and manage.	Kernel level threads are slower to create and manage.
2	Implementation is by a thread library at the user level.	Operating system supports creation of Kernel threads.
3	User level thread is generic and can run on any operating system.	Kernel level thread is specific to the operating system.
4	Multi-threaded application cannot take advantage of multiprocessing.	Kernel routines themselves can be multithreaded.

Process

A process is a program at the time of execution.

Differences between Process and Program

Process	**Program**
Process is a dynamic object	Program is a static object
Process is sequence of instruction execution	Program is a sequence of instructions
Process loaded in to main memory	Program loaded into secondary storage devices
Time span of process is limited	Time span of program is unlimited
Process is a active entity	Program is a passive entity

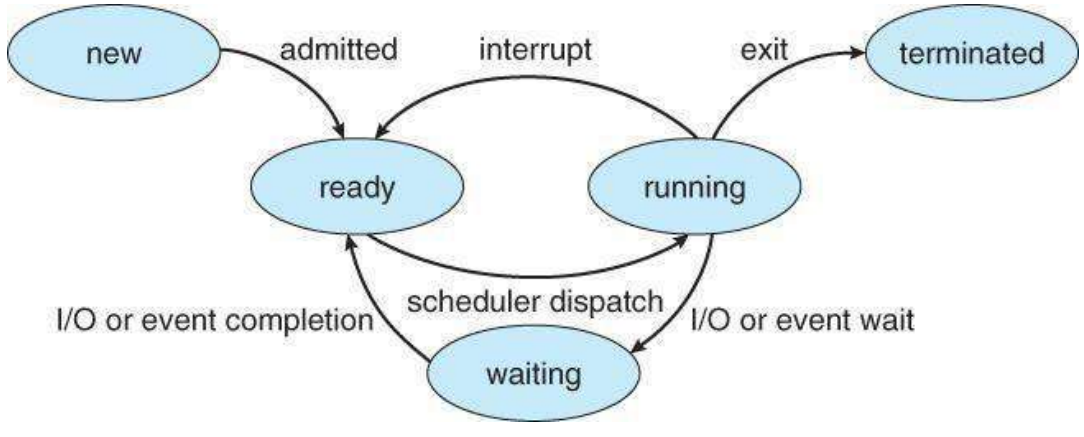
Process States

When a process executed, it changes the state, generally the state of process is determined by the current activity of the process. Each process may be in one of the following states:

1. **New** : The process is beingcreated.
2. **Running** : The process is beingexecuted.
3. **Waiting** : The process is waiting for some event tooccur.
4. **Ready** : The process is waiting to be assigned to a processor.
5. **Terminated** : The Process has finishedexecution.

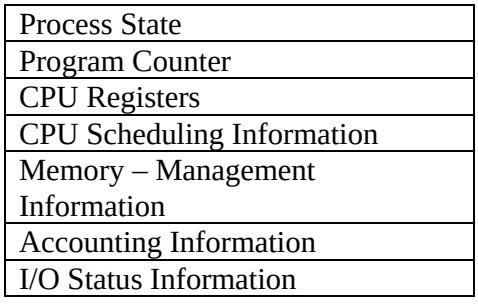
Only one process can be running in any processor at any time, But many process may be in ready and waiting states. The ready processes are loaded into a "ready queue".

Diagram of process state



- Each process is represented in the operating System by a **Process Control Block.**
- It is also called **Task Control Block.**
- It contains many pieces of information associated with a specific Process.

Fig: Process Control Block

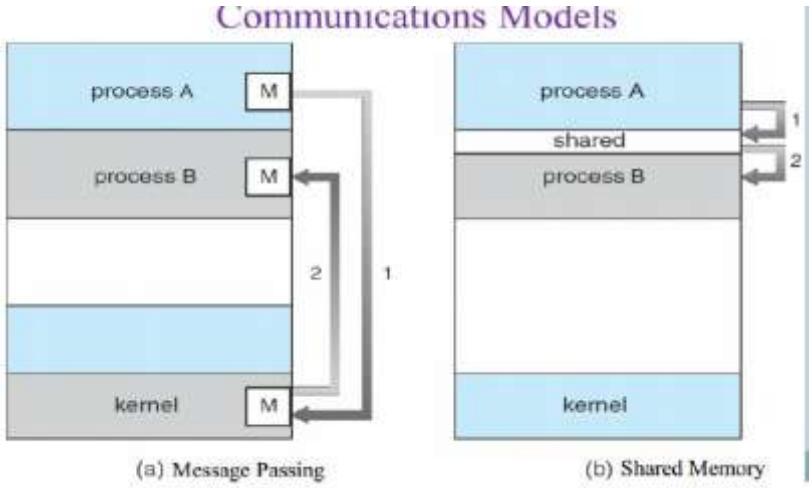


Process Control Block

1. **ProcessState** : The State may be new, ready, running, and waiting,Terminated…
2. **ProgramCounter** : indicates the Address of the next Instruction to be executed.
3. **CPUregisters** : registers include accumulators, stack pointers, General purpose Registers….
4. **CPU-SchedulingInfo** : includes a process pointer, pointers to schedulingQueues,other scheduling parametersetc.
5. **Memory management Info**: includes page tables, segmentation tables, value of base and limit registers.
6. **AccountingInformation:** includes amount of CPU used, time limits, Jobs(or)Process numbers.
7. **I/O StatusInformation**: Includes the list of I/O Devices Allocated to theprocesses, list of open files.

Inter Process communication:

Inter-process communication (IPC) is a fundamental mechanism that allows processes to exchange data and synchronize their actions in a computing system. When multiple processes run concurrently within an operating system, they can either be **independent** (not affected by other processes) or **cooperating** (interact with other executing processes). IPC facilitates resource and data sharing between these processes without interference.

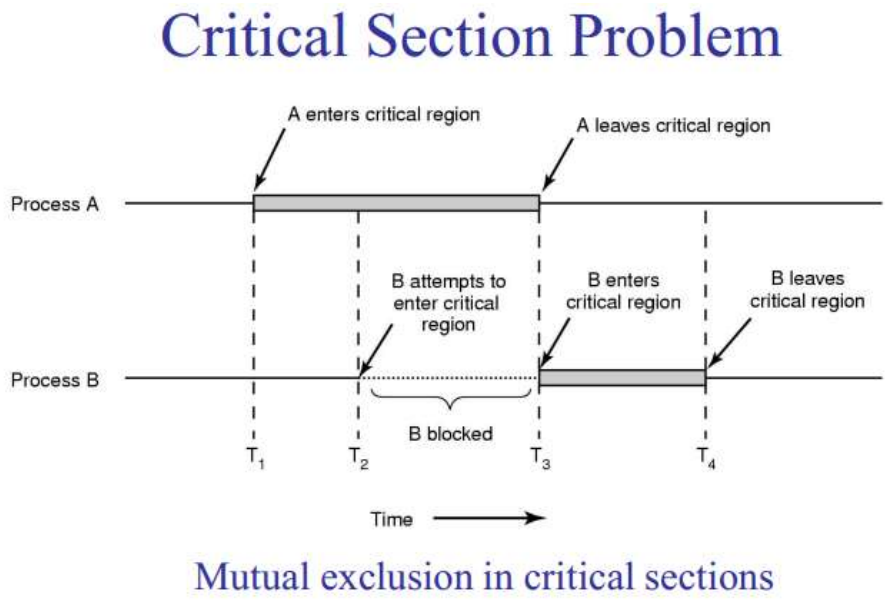


There are two primary methods of IPC:

1. **Shared Memory Method**:
   - In this approach, processes share a common memory region (referred to as **shared memory**).
   - One process writes data into this shared memory, and other processes can read from or write to it.
   - Example: The Producer-Consumer problem**,** where a producer generates items and stores them in a shared buffer, while a consumer retrieves items from the same buffer for consumption.
   - The shared memory method is efficient but requires careful synchronization to avoid race conditions.
2. **Message Passing**:
   - Processes communicate by sending messages to each other.
   - These messages can contain data or control information.
   - Example: A process sends a message to another process requesting specific data or notifying it about an event.
   - Message passing is more flexible than shared memory but may involve higher overhead due to message copying and synchronization[1].

Process synchronization refers to the idea that multiple processes are to join up or handshake at a certain point, in order to reach an agreement or commit to a certain sequence of action. Coordination of simultaneous processes to complete a task is known as process synchronization.

The Critical Section Problem



Consider a system , assume that it consisting of n processes. Each process having a segment of code. This segment of code is said to be critical section.

E.G: Railway Reservation System.

Two persons from different stations want to reserve their tickets, the train number, destination is common, the two persons try to get the reservation at the same time. Unfortunately, the available berths are only one; both are trying for that berth. It is also called the critical section problem. Solution is when one process is executing in its critical section, no other process is to be allowed to execute in its critical section.

What is a Critical Section Problem?

The Critical Section Problem is a Code Snippet. This code snippet contains a few variables. These variables can be accessed by a few processes. There is a condition for these processes.
The condition is that only one process can only enter the critical section. Remaining Processes which are interested to enter the critical section have to wait for the process to complete its work and then enter the critical section.

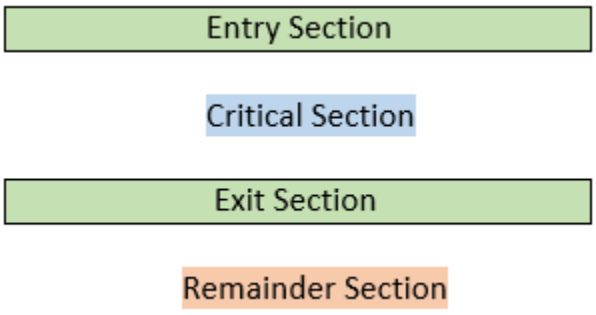


Figure: Critical Section Representation

The critical section problem is to design a protocol that the processes can use to cooperate. Each process must request permission to enter its critical section. The section of code implementing this request is the **entry section**. The critical section may be followed by an **exit section**. The remaining code is the **remainder section.**

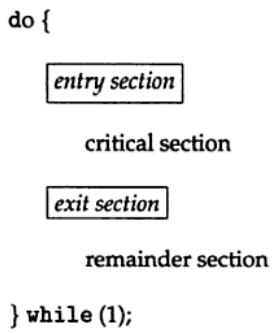


Figure General structure of a typical process P_i.

Problems in Critical Section Problems

There may be a state where one or more processes try to enter the critical state. After multiple processes enter the Critical Section, the second process try to access variable which already accessed by the first process.

Explanation

Suppose there is a variable which is also known as shared variable. Let us define that shared variable.

Here, x is the shared variable.

```
int x = 10;
```

Process 1

```
// Process 1
int s = 10;
int u = 20;
x = s + u;  //x=30
```

Process 2

```
// Process 2
int s = 10;
int u = 20;
x = s - u;  //x=-10
```

If both the processes occur at the same time, then the compiler would be in a confusion to choose which variable value i.e. -10 or 30. This state faced by the variable x is Data Inconsistency. These problems can also be solved by **Hardware Locks.**

To, prevent such kind of problems can also be solved by ***Hardware solutions*** *named* ***Semaphores****.*

Sleep and Wakeup
Producer-Consumer Problem

- Two process share a common, fixed sized buffer.
- Suppose one process, producer, is generating information that the second process, consumer, is using.
- Their speed may be mismatched, if producer insert item rapidly, the buffer will full and go to the sleep until consumer consumes some item, while consumer consumes rapidly, the buffer will empty and go to sleep until producer put something in the buffer.

Semaphores

The Semaphore is just a normal integer. The Semaphore cannot be negative. The least value for a Semaphore is zero (0). The Maximum value of a Semaphore can be anything. The Semaphores usually have two operations. The two operations have the capability to decide the values of the semaphores.

The two Semaphore Operations are:

1. Wait ( )
2. Signal ( )

Wait Semaphore Operation

The Wait Operation is used for deciding the condition for the process to enter the critical state or wait for execution of process. Here, the wait operation has many different names. The different names are:

1. Sleep Operation
2. Down Operation
3. Decrease Operation
4. P Function (most important alias name for wait operation)

Basic Algorithm of P Function or Wait Operation

P (Semaphore value)
{
Allow the process to enter **if** the value of Semaphore is greater than zero or positive.
Do not allow the process **if** the value of Semaphore is less than zero or zero.
Decrement the Semaphore value **if** the Process leaves the Critical State.
}

Signal Semaphore Operation

The Signal Semaphore Operation is used to update the value of Semaphore. The Semaphore value is updated when the new processes are ready to enter the Critical Section.

The Signal Operation is also known as:

1. Wake up Operation
2. Up Operation
3. Increase Operation
4. V Function (most important alias name for signal operation)

Basic Algorithm of V Function or Signal Operation

V (Semaphore value)
{
If the process goes out of the critical section then add 1 to the semaphore value
Else keep calm until process exits
}

Types of Semaphore: There are two types of semaphore:

1.Counting Semaphores
These are integer value semaphores and have an unrestricted value domain. These semaphores are used to coordinate there source access, where the semaphore count is the number of available resources. If there sources are added, semaphore count automatically incremented and if the resources are removed, the count is decremented.
2.BinarySemaphores

The binary semaphores are like counting semaphores but their value is restricted to 0and 1.The wait operation only works when the semaphore is 1 and the signal operation succeeds when semaphore is 0.It is sometimes easier to implement binary semaphores than counting semaphores.

Monitor

Monitor in OS (operating system) is a synchronization construct that enables multiple processes or threads to coordinate actions and ensures that they are not interfering with each other or producing unexpected results. Also, it ensures that only one thread is executed at a critical code section.

Syntax:

The syntax of the monitor may be used as:

```
monitor  {

   //shared variable declarations
   data variables;
   Procedure P1() { ... }
   Procedure P2() { ... }
   .
   .
   .
   Procedure Pn() { ... }
   Initialization Code() { ... }
}
```

Advantages

- Monitors are less difficult to implement than semaphores.
- Monitors may overcome the timing errors that occur when semaphores are used.
- Monitors are a collection of procedures and condition variables that are combined in a special type of module.

Disadvantages

- Monitors must be implemented into the programming language.
- The compiler should generate code for them.
- It gives the compiler the additional burden of knowing what operating system features is available for controlling access to crucial sections in concurrent processes.

Various Times Related To Process-

- Arrival time
- Waiting time
- Response time
- Burst time
- Completion time
- Turn Around Time

1. Arrival Time-

Arrival time is the point of time at which a process enters the ready queue.

2. Waiting Time-

Waiting time is the amount of time spent by a process waiting in the ready queue for getting the CPU.

Waiting time = Turn Around time – Burst time

3. Response Time-

Response time is the amount of time after which a process gets the CPU for the first time after entering the ready queue.

Response Time = Time at which process first gets the CPU – Arrival time

4. Burst Time-

- Burst time is the amount of time required by a process for executing on CPU.
- It is also called as execution time or running time.
- Burst time of a process can not be known in advance before executing the process.
- It can be known only after the process has executed.

5. Completion Time

Completion time is the point of time at which a process completes its execution on the CPU and takes exit from the system.

It is also called as exit time.

6. Turn Around Time-

Turn Around time is the total amount of time spent by a process in the system.

When present in the system, a process is either waiting in the ready queue for getting the CPU or it is executing on the CPU.

Turn Around time = Burst time + Waiting time

OR

Turn Around time = Completion time – Arrival time

Process Scheduling Algorithms

CPU scheduling is the process of deciding which process will own the CPU to use while another process is suspended. The main function of the CPU scheduling is to ensure that whenever the CPU remains idle, the OS has at least selected one of the processes available in the ready-to-use line.

Objectives of Process Scheduling Algorithm:

- Utilization of CPU at maximum level. **Keep CPU as busy as possible**.
- **Allocation of CPU should be fair**.
- **Throughput should be Maximum**. i.e. Number of processes that complete their execution per time unit should be maximized.
- **Minimum turnaround time**, i.e. time taken by a process to finish execution should be the least.
- There should be a **minimum waiting time** and the process should not starve in the ready queue.
- **Minimum response time.** It means that the time when a process produces the first response should be as less as possible.

Terminologies CPU Scheduling Algorithm

- **Arrival Time:** Time at which the process arrives in the ready queue.
- **Completion Time:** Time at which process completes its execution.
- **Burst Time:** Time required by a process for CPU execution.
- **Turn Around Time:** Time Difference between completion time and arrival time.

Turn Around Time = Completion Time – Arrival Time

- **Waiting Time(W.T):** Time Difference between turn around time and burst time.
  Waiting Time = Turn Around Time – Burst Time

There are mainly two types of scheduling methods:

- **Preemptive Scheduling:** Preemptive scheduling is used when a process switches from running state to ready state or from the waiting state to the ready state.
- **Non-Preemptive Scheduling:** Non-Preemptive scheduling is used when a process terminates , or when a process switches from running state to waiting state.

1. First Come First Serve:

FCFS considered to be the simplest of all operating system scheduling algorithms. First come first serve scheduling algorithm states that the process that requests the CPU first is allocated the CPU first and is implemented by using FIFO queue.

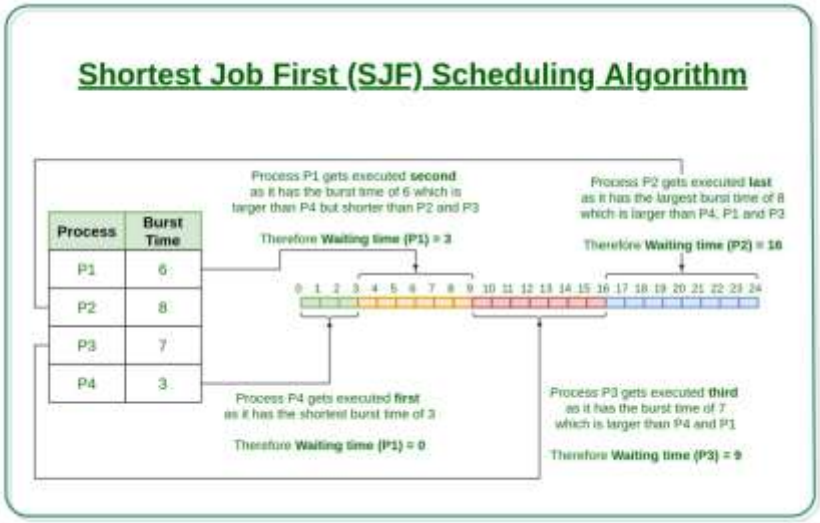
Characteristics of FCFS:
- FCFS supports non-preemptive and preemptive CPU scheduling algorithms.
- Tasks are always executed on a First-come, First-serve concept.
- FCFS is easy to implement and use.
- This algorithm is not much efficient in performance, and the wait time is quite high.

Advantages of FCFS:
- Easy to implement
- First come, first serve method

Disadvantages of FCFS:
- FCFS suffers from **Convoy effect**.
- The average waiting time is much higher than the other algorithms.
- FCFS is very simple and easy to implement and hence not much efficient.

2. Shortest Job First(SJF):

Shortest job first (SJF) is a scheduling process that selects the waiting process with the smallest execution time to execute next. Out of all the available processes, CPU is assigned to the process having smallest burst time. This scheduling method may or may not be preemptive. Significantly reduces the average waiting time for other processes waiting to be executed.



Characteristics of SJF:

- Shortest Job first has the advantage of having a minimum average waiting time among all operating system scheduling algorithms.
- It is associated with each task as a unit of time to complete.
- It may cause starvation if shorter processes keep coming. This problem can be solved using the concept of ageing.

Advantages of Shortest Job first:

- As SJF reduces the average waiting time thus, it is better than the first come first serve scheduling algorithm.
- SJF is generally used for long term scheduling

Disadvantages of SJF:

- One of the demerit SJF has is starvation.
- Many times it becomes complicated to predict the length of the upcoming CPU request

3. Priority Scheduling

- Out of all the available processes, CPU is assigned to the process having the highest priority.
- Priority Scheduling can be used in both preemptive and non-preemptive mode.

Advantages-

- It considers the priority of the processes and allows the important processes to run first.
- Priority scheduling in preemptive mode is best suited for real time operating system.

Disadvantages-

- Processes with lesser priority may starve for CPU.
- There is no idea of response time and waiting time.

4. Round Robin Scheduling

- CPU is assigned to the process on the basis of FCFS for a fixed amount of time.
- This fixed amount of time is called as time quantum or time slice.
- After the time quantum expires, the running process is preempted and sent to the ready queue.
- Then, the processor is assigned to the next arrived process.
- It is always preemptive in nature.

Note:
Thus, smaller value of time quantum is better in terms of response time.
Thus, higher value of time quantum is better in terms of number of context switch.

Advantages-

- It gives the best performance in terms of average response time.
- It is best suited for time sharing system, client server architecture and interactive system.

Disadvantages-

- It leads to starvation for processes with larger burst time as they have to repeat the cycle many times.
- Its performance heavily depends on time quantum.
- Priorities can not be set for the processes.